

Labo 04 — Le Dockerfile : construire ses propres images

Théorie — la recette, le contexte, le cache, les deux pièges qui coûtent le plus cher, et ce que change un moteur de build sans daemon.

Objectifs

- Comprendre ce qu'est le **build context** et pourquoi il détermine ce que vous pouvez copier.
- Connaître les instructions essentielles et ce que chacune produit.
- Distinguer `CMD` et `ENTRYPOINT`, forme *shell* et forme *exec*.
- Distinguer `ARG` et `ENV`.
- Ordonner un Dockerfile pour exploiter le **cache de construction**.

1. Le build context

```
podman build -t mon-api:1.0 .
```

Le `.` final n'est pas décoratif : c'est le **contexte de construction**, le dossier que le build a le droit de lire. Deux conséquences absolues :

- **Vous ne pouvez copier que ce qui est dans le contexte.** `COPY ../secrets/cle.pem .` échoue toujours : le fichier est hors du périmètre. Pas de contournement.
- **Tout le dossier fait partie du contexte**, y compris `.git/`, `node_modules/`, `target/`, les logs et la configuration locale. Un `COPY . .` embarque tout cela dans l'image.

PODMAN

Chez Docker, le client **empaquette** le contexte dans une archive et l'**envoie** au daemon — c'est la ligne `transferring context: 900MB` qui fait durer les builds Angular. Chez Podman, le build est fait par **Buildah**, intégré dans le même processus, qui lit le dossier directement : pas d'archive, pas de transfert. Le contexte reste la frontière de ce qui est copiable, et `.dockerignore` reste indispensable — non pour la vitesse, mais pour ce qui finit **dans l'image**. Buildah accepte aussi les noms neutres `Containerfile` et `.containerignore`.

Le fichier `.dockerignore`, à la racine du contexte, exclut ce qui ne doit pas entrer :

```
.git
node_modules
target
.env
```

PIÈGE

Sans `.dockerignore`, un `COPY . .` embarque les secrets locaux et le `.git` complet **dans l'image finale**, pour quiconque la télécharge. C'est une fuite de données classique.

Le Dockerfile lui-même peut être ailleurs : `-f docker/api.Dockerfile` le désigne.

2. Les instructions essentielles

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine # image de base – toujours la 1re instruction
LABEL org.opencontainers.image.source="https://git.masociete.be/paiement/api"
WORKDIR /app # crée et se place dans le dossier
COPY target/api.jar /app/api.jar # copie depuis le contexte vers l'image
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75" # variable présente à l'exécution
EXPOSE 8080 # documentation : ne publie AUCUN port
USER 1000:1000 # ne pas tourner en root
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar /app/api.jar"]
```

Instruction	Rôle	Couche de fichiers ?
FROM	Point de départ	oui (celles de la base)
RUN	Exécute une commande au build	oui
COPY / ADD	Copie depuis le contexte	oui
WORKDIR, ENV, USER, EXPOSE, LABEL	Métadonnées	non (0B)
CMD, ENTRYPOINT	Ce qui s'exécute au <code>run</code>	non
ARG	Variable du build seulement	non

Trois précisions. **COPY plutôt qu'ADD** : `ADD` décompresse les archives et télécharge les URL, deux comportements implicites. **EXPOSE ne publie rien** : c'est `-p` qui publie (labo 07). **RUN s'exécute au build** : `RUN java -jar api.jar` lancerait l'application pendant la construction.

À RETENIR

Écrivez le `FROM` en entier : `docker.io/library/eclipse-temurin:21-jre-alpine`. Un `FROM eclipse-temurin:...` dépend de la configuration de la machine qui construit (labo 02) ; en entreprise, ce sera `registry.interne/socle/...`.

3. CMD contre ENTRYPOINT

Les deux définissent ce qui tourne au démarrage. Leur différence est leur rapport aux arguments de `podman run` :

- CMD** est une valeur **par défaut, remplaçable**. `podman run mon-image autre-commande` ignore le `CMD`.
- ENTRYPOINT** est le programme **fixe**. Les arguments de `podman run` lui sont **ajoutés**.

```
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
CMD ["--spring.profiles.active=prod"]
```

`podman run api` lance `java -jar /app/api.jar --spring.profiles.active=prod` ; `podman run api --spring.profiles.active=dev` lance la même chose avec le profil `dev`. C'est le motif standard : `ENTRYPOINT` fixe le programme, `CMD` fournit les arguments par défaut ; `podman run --entrypoint sh -it mon-image` reste la porte de sortie pour déboguer.

→ SPRING BOOT

Les arguments passés après le JAR (`--spring.profiles.active=dev` , `--server.port=9090`) sont lus par Spring comme des propriétés prioritaires sur `application.yml` . D'où la commodité du duo `ENTRYPOINT` + `CMD` : même image, un argument différent par environnement. Le labo 08 montrera que les variables d'environnement sont encore préférables.

4. Forme *shell* et forme *exec*

Toute commande peut s'écrire de deux façons, et ce n'est pas une question de style :

```
CMD java -jar /app/api.jar # forme SHELL -> /bin/sh -c "java -jar ..."  
CMD ["java","-jar","/app/api.jar"] # forme EXEC -> java devient PID 1
```

En forme *exec*, l'application **est** le PID 1 : elle reçoit `SIGTERM` et s'arrête proprement. En forme *shell*, un `/bin/sh` s'intercale — le problème du labo 03 : un shell qui reste PID 1 ne relaie pas `SIGTERM`, l'application ne le reçoit jamais, `stop` attend dix secondes et tue tout.

Le mot important est **reste**. Le comportement dépend de l'implémentation du shell :

Cas	PID 1	podman stop
Forme <i>exec</i>	votre application	propre, code 143
Forme <i>shell</i> , commande simple, base Alpine (busybox)	votre application (le shell s'efface)	propre, code 143
Forme <i>shell</i> , commande simple, base Debian/Ubuntu (dash)	<code>/bin/sh</code>	10 s puis code 137
Forme <i>shell</i> avec un tube, un <code>&</code> , un <code>;</code>	<code>/bin/sh</code>	10 s puis code 137
Script d'entrée qui lance l'appli sans <code>exec</code>	<code>/bin/sh</code>	10 s puis code 137

→ LINUX / SHELL

`/bin/sh` n'est pas un programme unique : `dash` sur Debian et Ubuntu, `ash` de busybox sur Alpine. Certains, quand la commande est la *dernière* du script, s'auto-remplacent par elle (un `exec` implicite) ; d'autres créent un enfant et attendent. D'où un Dockerfile qui s'arrête proprement sur Alpine et pas sur Debian.

À RETENIR

Écrivez toujours la forme *exec*, avec des guillemets doubles JSON. Si vous devez passer par un shell, écrivez-le explicitement **et** utilisez `exec` : `ENTRYPOINT ["sh","-c","exec java $JAVA_OPTS -jar /app/api.jar"]` .

Conséquence moins connue : en forme *exec*, `$JAVA_OPTS` , `&&` , `|` et `>` ne sont **pas** interprétés — il n'y a pas de shell pour le faire.

5. ARG contre ENV

```
ARG VERSION=1.0 # disponible pendant le build uniquement  
ENV APP_VERSION=${VERSION} # persiste dans l'image et dans les conteneurs
```

	ARG	ENV
Visible pendant le build	oui	oui
Présent dans l'image finale	non	oui
Modifiable au build	<code>--build-arg VERSION=2.0</code>	non
Modifiable à l'exécution	non	<code>podman run -e APP_VERSION=...</code>

PIÈGE

« ARG disparaît de l'image » ne veut **pas** dire « ARG est sûr pour un secret » : la valeur reste visible dans `podman history` et dans le cache de build. Un mot de passe passé en `--build-arg` est une fuite (labo 08).

6. Le cache de construction

Le moteur traite les instructions dans l'ordre et met chaque résultat en cache. Pour chacune, il se demande : « ai-je déjà exécuté celle-ci, à partir de la même couche précédente ? » Si oui, il réutilise (`--> Using cache`). Sinon il l'exécute — **et invalide tout ce qui suit**. L'invalidation vient d'une modification du texte de l'instruction, du **contenu** des fichiers copiés (`COPY` / `ADD`), ou d'une instruction précédente invalidée, en cascade. D'où la règle d'or : **du plus stable au plus volatil**.

```
# MAUVAIS : le code change à chaque commit, donc tout est reconstruit
COPY . /app
RUN mvn dependency:go-offline

# BON : les dépendances ne sont retéléchargées que si pom.xml change
COPY pom.xml /app/
RUN mvn dependency:go-offline
COPY src /app/src
```

Même raisonnement pour Angular : `COPY package*.json` puis `npm ci`, et seulement ensuite `COPY . .`. Le gain se compte en minutes par build de CI — et, puisqu'une couche inchangée n'est pas retransférée au `push` (labo 02), en temps de déploiement. Pour tout reconstruire : `--no-cache`.

7. Écrire un RUN correct

```
# MAUVAIS : trois couches, un cache d'apt de 40 Mo embarqué dans l'image
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*

# BON : une seule couche, nettoyage effectif
RUN apt-get update \
&& apt-get install -y --no-install-recommends curl \
&& rm -rf /var/lib/apt/lists/*
```

Le nettoyage doit avoir lieu **dans le même RUN** que l'installation : une suppression ultérieure masque sans retirer (labo 02). Et `apt-get update` ne doit jamais être seul dans sa couche : mis en cache pendant des semaines, il servirait des index périmés — le problème du *cache busting*.

8. En entreprise

Le Dockerfile d'un back Spring Boot ressemble à ceci (version simple ; la version multi-stage arrive au labo 05) :

```
FROM registry.interne/socle/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY target/api-*.jar app.jar
EXPOSE 8080
USER 1000:1000
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Points à noter : image **JRE** et non JDK, tirée du registry interne, `USER` non-root, forme exec, JAR copié depuis `target/` — donc le build Maven a eu lieu **avant**, en CI ; c'est la limite que le multi-stage va lever. Côté Angular, on ne conteneurise jamais `ng serve`, mais le résultat de `ng build` servi par nginx. La même recette est construite par Docker en CI et par Podman sur votre poste : un Dockerfile est un standard.

À retenir

- Le `.` de `build .` désigne le **contexte** : ce qui est copiable, et ce qu'un `COPY . .` embarque. `.dockerignore` est obligatoire — même sans transfert, avec Podman.
- `EXPOSE` documente, `-p` publie. `RUN` s'exécute au build, `CMD` / `ENTRYPOINT` à l'exécution ; `ENTRYPOINT` fixe le programme, `CMD` fournit des arguments remplaçables.
- Forme exec `["prog", "arg"]` : l'application est PID 1 et reçoit `SIGTERM`. Forme *shell* : ça dépend du shell de l'image de base — donc non. `ARG` vit pendant le build, `ENV` persiste dans l'image — aucun des deux ne convient à un secret.
- Ordonnez du plus stable au plus volatil ; toute instruction invalidée invalide les suivantes.
- Installation et nettoyage dans le **même** `RUN`, sinon les fichiers restent.

Vocabulaire

build context : dossier que le build a le droit de lire. — `.dockerignore` / `.containerignore` : exclusions du contexte. — **Containerfile** : nom neutre du Dockerfile chez Podman. — **Buildah** : moteur de build de Podman. — **forme exec / shell** : deux écritures de `CMD` / `ENTRYPOINT`. — **cache busting** : invalidation volontaire du cache. — **image de base** : image citée par `FROM`.